

FRAMEWORK PLC ESP32 · BORNEUNIVERSELLE

PLC Studio

Guide d'utilisation : configurer, programmer, simuler et générer un projet d'automate sans écrire le code à la main.

1 Projet

2 Équipements

3 Câblage et variables

4 Logique (grafcet)

5 Écrans opérateur

6 Simulation

7 Génération

- | | |
|-----------------------------------|-------------------------------------|
| 1 À quoi sert PLC Studio | 6 Aide-mémoire de la logique |
| 2 Installer et lancer l'outil | 7 Règles appliquées automatiquement |
| 3 Prise en main de l'interface | 8 Limites connues et dépannage |
| 4 Les sept étapes d'un projet | 9 Pour les développeurs |
| 5 Mettre le projet sur l'automate | |

Outil situé dans `tools/plc-studio/` du dépôt PLC · Projets enregistrés dans `projets/` · Documentation de référence : `tools/plc-studio/README.md` · Version du document : octobre 2026

1. À quoi sert PLC Studio

Le firmware de l'automate (ESP32, framework **BorneUniverselle**) attend trois éléments propres à chaque machine :

- `data/config.json` : réseau WiFi, options, et la liste de tous les **nœuds** (entrées, sorties, registres Modbus, variables internes) avec leurs identifiants ;
- `data/interface.json` : les pages, sections et widgets de l'écran opérateur (navigateur ou tablette) ;
- une **classe C++ d'application** dérivée de `BusinessLogic` (aujourd'hui `src/RessortRoyal2/`) qui contient la logique de la machine.

Écrire ces fichiers à la main est long et fragile : un identifiant de nœud mal calculé, un nom trop long ou un widget qui pointe vers un nœud absent bloquent l'automate au démarrage. **PLC Studio** est une application web locale qui produit ces trois éléments, cohérents entre eux, à partir d'une description visuelle de la machine :

- 1 vous choisissez les **équipements** (automate Kincony, modules d'E/S Waveshare, servo SureServo) ;
- 2 vous indiquez **ce qui est branché** sur chaque voie et créez les variables internes ;
- 3 vous dessinez la **logique en grafcet** et réglez les modes de marche (arrêt d'urgence, marche/arrêt, prise d'origine, jog) ;
- 4 vous composez les **écrans opérateur** sur l'interface réelle de l'automate ;
- 5 vous **simulez** le fonctionnement, puis vous **générez** et installez les fichiers dans le dépôt.

Bon à savoir Le simulateur exécute la logique exactement comme la classe C++ générée (même cycle de 10 ms, même ordre de traitement) : ce qui fonctionne en simulation fonctionne sur l'automate, au matériel près.

2. Installer et lancer l'outil

Prérequis

- **Node.js 18 ou plus récent** (version « LTS » sur <https://nodejs.org>). Aucune autre installation : l'outil n'a aucune dépendance `npm` à télécharger.
- Une copie du dépôt PLC sur le poste (le dossier qui contient `platformio.ini`, `data/` et `src/`).
- Un navigateur récent (Chrome, Edge ou Firefox).
- Pour téléverser sur l'automate : **PlatformIO** (extension VS Code ou ligne de commande), comme pour le firmware actuel.

Pour vérifier Node.js, ouvrez un terminal (PowerShell sous Windows) et tapez :

```
node --version      # doit afficher v18.x ou plus
```

Lancer PLC Studio

Depuis la racine du dépôt PLC :

```
# Windows (PowerShell)
cd C:\chemin\vers\PLC
node tools\plc-studio\server\server.js

# macOS / Linux
cd ~/chemin/vers/PLC
node tools/plc-studio/server/server.js
```

Variante équivalente : `cd tools/plc-studio` puis `npm start` .

Le terminal affiche l'adresse à ouvrir. Ouvrez ensuite dans le navigateur :

```
http://localhost:8090
```

Laissez le terminal ouvert pendant l'utilisation ; **Ctrl+C** dans le terminal arrête l'outil.

Options de lancement

Option	Défaut	Rôle
<code>--port <n></code>	8090	Port HTTP (à changer si 8090 est déjà utilisé, ex. <code>--port 8091</code>).
<code>--root <dossier></code>	racine du dépôt	Dépôt PLC à lire et à écrire (utile si l'outil est lancé depuis un autre endroit).
<code>--host <adresse></code>	127.0.0.1	Interface d'écoute. Par défaut, seul le poste local peut ouvrir l'outil.

Sécurité L'outil n'a pas d'authentification et lit les fichiers de configuration (mots de passe WiFi compris). Gardez l'écoute locale par défaut ; n'utilisez `--host 0.0.0.0` que sur un réseau de confiance.

Où sont les fichiers ?

Emplacement	Contenu
projets/<Nom>.plc.json	Le projet PLC Studio (équipements, variables, grafcet, écrans). C'est le fichier à conserver et à versionner.
projets/<Nom>/generation/	Fichiers produits par « Exporter » (sans toucher au firmware).
tools/plc-studio/.sauvegardes/	Copies des fichiers remplacés lors d'une « Installation », classées par date.
projets/DemoPresse.plc.json	Projet d'exemple complet (presse avec servo) pour découvrir l'outil.

3. Prise en main de l'interface

- **Barre du haut** : **Ouvrir...** (projets de `projets/`), **Nouveau** (nom du projet) et **Enregistrer** (Ctrl+S). Après le premier enregistrement, chaque modification est enregistrée automatiquement.
- **Colonne de gauche** : les sept étapes. Un badge indique le nombre d'erreurs ou d'avertissements par étape ; le bas de la colonne résume l'état du projet (« Projet valide », nombre de nœuds, application actuellement installée).
- **Navigation libre** : on peut revenir à n'importe quelle étape à tout moment (par exemple ajouter une variable pendant l'écriture du grafcet) ; tout est recalculé et revérifié en direct.
- Chaque message « À corriger » propose un lien → **corriger** qui mène à l'endroit concerné.

Pour découvrir Lancez l'outil, cliquez sur **Ouvrir...** puis choisissez **DemoPresse** : toutes les étapes sont remplies et la simulation est prête à tourner.

4. Les sept étapes d'un projet

Étape 1 — Projet

Nom du projet (il devient le nom de la classe C++ et du fichier de paramètres `data/<Nom>.json`), **nom réseau** (mDNS, sans espace), description, réseaux **WiFi** (10 au maximum, en client ou en point d'accès), options du journal, mot de passe de l'éditeur de fichiers et URL de mise à jour.

Mode d'exécution : *Avec écran (HMI)* — la logique ne tourne que lorsqu'un navigateur est connecté (comportement actuel du firmware) ; *Autonome (headless)* — la logique tourne en permanence. Ce choix est écrit dans `config.json` (`execution_mode`), mais le firmware ne le lit pas encore (voir §8).

PLC Studio Projet : DemoPresse Enregistré dans projets/DemoPresse.plc.json Ouvrir... Nouveau Enregistrer

1 Projet

2 Équipements

3 Câblage et variables 1

4 Logique (grafcet)

5 Écrans opérateur

6 Simulation

7 Génération

Projet valide
58 nœuds
Application actuelle :
RessortRoyal2

Projet
Identité du projet, mode de fonctionnement et accès réseau de l'automate.

Identité

Nom du projet	Nom réseau (mDNS)	Description
DemoPresse	demopresse	

Nom de la classe C++ générée (src/<Nom>/) et du fichier data/<Nom>.json.

Clé « name » de config.json, sans espace. L'automate répond sur http://<nom>.local

Mode d'exécution
Indique si la logique doit attendre qu'un écran (navigateur) soit connecté pour tourner.

☒ **Avec écran (HMI)**
La logique ne tourne que lorsqu'un navigateur est connecté. C'est le comportement actuel du firmware.

☐ **Autonome (headless)**
La logique tourne en permanence, avec ou sans écran connecté.

Ce réglage est écrit dans config.json (clé « execution_mode »), mais le firmware actuel ne le lit pas encore : aujourd'hui, la logique ne tourne que si un écran est connecté (audit CR-2). La correction de main.cpp est prévue plus tard.

Options

☒ Afficher l'état machine à l'écran	☐ Afficher l'horloge à l'écran	☒ Journal de diagnostic (fichier)	☐ Effacer le journal au démarrage
☐ Notifications de succès à l'écran	☒ Notifications d'information à l'écran	☐ Historique des transitions dans le journal	

Mot de passe de l'éditeur de fichiers **URL de mise à jour (OTA)**

Vérifié uniquement par le navigateur. 79 caractères maximum.

Étape 1 — identité du projet, réseau et mode d'exécution.

Étape 2 — Équipements

Ajoutez les équipements depuis le catalogue : automate **Kincony KC868-A8S**, modules Modbus **Waveshare** (8 E/S TOR, 16 sorties, 8 entrées analogiques) et servo **SureServo 2**. Pour chaque module Modbus, réglez l'adresse (1 à 247). L'automate est toujours placé en premier : c'est lui qui porte le bus RS485 (sinon le firmware bloque au démarrage). Un seul servo par projet.

PLC Studio

Projet : DemoPresse

Enregistré dans projets/DemoPresse.plc.json

Ouvrir...

Nouveau

Enregistrer

1 Projet

2 Équipements

3 Câblage et variables 1

4 Logique (grafcet)

5 Écrans opérateur

6 Simulation

7 Génération

Projet valide

58 nœuds

Application actuelle : RessortRoyal2

Équipements

Sélectionnez les équipements avec lesquels le projet interagira. Vous câblerez leurs voies à l'étape suivante.

Équipements du projet

L'automate est toujours déclaré en premier dans config.json (il porte le bus RS485).

Automate Kincony KC868-A8S Automate

7 voie(s) câblée(s) Câbler... Retirer

Nom

Automate

☒ Horloge DS3231 installée

Si décoché, l'outil génère une variante du fichier matériel sans la clé RTC (sinon le démarrage bloque quand l'horloge est absente).

Entrées TOR : 8 · Sorties relais : 8 · Sortie directe GPIO2 : 1

Module ES Waveshare Modbus RTU IO 8CH E/S Modbus

3 voie(s) câblée(s) Câbler... Retirer

Nom

Module ES

Adresse Modbus

1

Sert à nommer les sections de config.json.

Entre 1 et 247, unique sur le bus.

Entrées TOR : 8 · Sorties TOR : 8

Servo Servo AutomationDirect SureServo 2 Servo Modbus

Câbler... Retirer

Nom

Servo

Adresse Modbus

127

Impulsions par unité

1

Course maximale (unités)

1000000

Sert à nommer les sections de config.json.

Entre 1 et 247, unique sur le bus.

1 = travail direct en impulsions (PUU).

Butée logicielle : toute cible au-delà est refusée (avec 1 impulsion par unité, la course est en impulsions).

Position négative

Invalide l'origine

Étape 2 — équipements du projet et catalogue.

Étape 3 — Câblage et variables

Pour chaque voie, tapez simplement **ce qui est branché** (« Arrêt d’urgence OK », « Vérin sorti »...) : le **symbole** utilisé dans la logique est proposé automatiquement (modifiable). Cochez **Inversée** pour un contact normalement fermé et **Écran** pour l’afficher. Les signaux du servo (servo ON, arrêt immédiat, reset alarmes) se câblent sur des sorties de l’automate.

Créez ensuite les **variables internes** dont la logique a besoin :

Type	Usage
Commande	Bouton ou interrupteur à l’écran, lu par la logique (Départ, Arrêt, Acquitter...).
Paramètre	Consigne réglable à l’écran et conservée après redémarrage (bornes min/max, valeur initiale).
Indicateur	Valeur calculée par la logique et affichée à l’écran (compteur, défaut...).
Mémoire	Variable de travail de la logique, sans affichage.

PLC Studio

Projet : DemoPress

Enregistré dans projets/DemoPress.plc.json

Ouvrir...

Nouveau

Enregistrer

1

Projet

2

Équipements

3

Câblage et variables

4

Logique (grafcet)

5

Écrans opérateur

6

Simulation

7

Génération

Projet valide

58 nœuds

Application actuelle : RessortRoyal2

Câblage et variables

Indiquez ce qui est branché sur chaque voie, puis créez les variables internes dont la logique a besoin. Vous pouvez revenir ici à tout moment, y compris pendant l’écriture de la logique.

À corriger

Attention « Voyant marche » : GPIO2 est le buzzer de la carte ; main.cpp le remet à 0 à chaque connexion WIFI (turnOffBuzzer).

Signaux à câbler

Ces signaux sont pilotés automatiquement (bloc servo). Choisissez la sortie sur laquelle chacun est branché.

Signal	Sortie	Symbole
Servo : Servo ON	Automate · Sorties relais · Y5	Servo_servoOn
Servo : Arrêt immédiat	Automate · Sorties relais · Y6	Servo_immediateStop
Servo : Reset alarmes	Automate · Sorties relais · Y7	Servo_alarmsReset

Automate Kincony KC868-A85

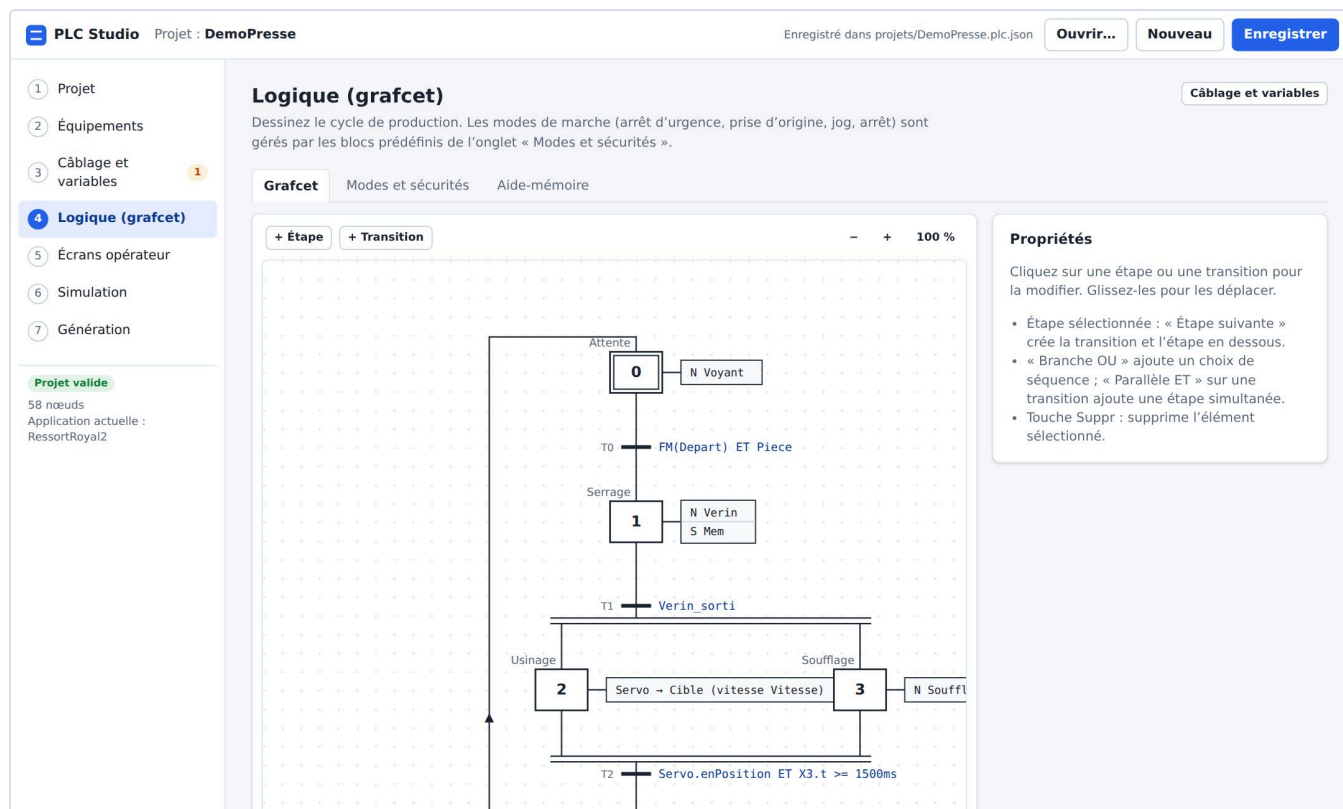
Entrées TOR — entrées booléens

Voie	Ce qui est branché	Symbole	Inversée	Écran
X1	AU OK	AU_OK	☐	☒
X2	Départ cycle	Depart	☐	☒
X3	— libre —			
X4	— libre —			
X5	— libre —			
X6	— libre —			

Étape 3 — câblage des voies et signaux du servo.

Étape 4 — Logique (grafcet)

L'onglet **Grafcet** est un éditeur visuel : **Créer un grafcet de départ**, puis sélectionnez une étape et utilisez **Étape suivante**, **Branche OU** (choix de séquence) ou, sur une transition, **Parallèle ET** (étapes simultanées). Les éléments se déplacent à la souris ; la touche **Suppr** supprime l'élément sélectionné. Dans le panneau de droite, écrivez les **réceptivités** (suggestions de symboles pendant la frappe, erreurs signalées immédiatement) et ajoutez les **actions** de chaque étape (voir l'aide-mémoire §6).



Étape 4 — grafcet avec divergence ET et actions sur le servo.

L'onglet **Modes et sécurités** règle les blocs prédéfinis qui entourent le grafcet :

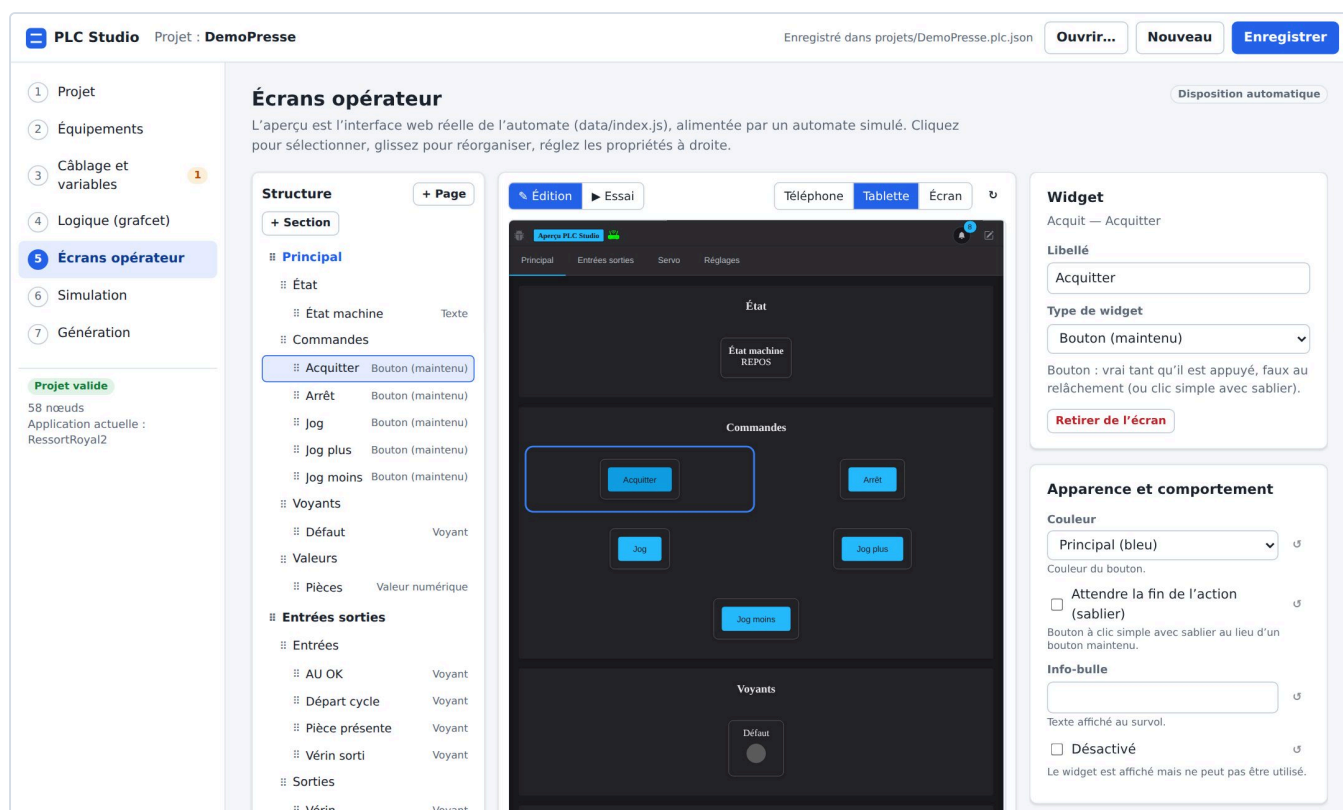
- **Arrêt d'urgence** : condition d'urgence (au minimum, ex. `NON AU_0K`) et condition d'acquiescement. En urgence, chaque sortie prend son état de repli, le servo est arrêté et le grafcet remis à zéro ; les alarmes du servo déclenchent aussi l'urgence.
- **Marche / arrêt** : condition de mise en marche (automatique si vide) et condition d'arrêt.
- **Servo** : prise d'origine (automatique avant le premier cycle, sur condition...), couple maximal, mode manuel (jog) depuis l'écran avec ses commandes et sa vitesse.

Étape 5 — Écrans opérateur

L'aperçu central est **l'interface web réelle de l'automate** (les mêmes fichiers que sur la carte), branchée sur un automate simulé : ce que vous voyez est ce qui s'affichera sur la tablette.

- **Structure** (à gauche) : pages > sections > widgets, réordonnables par glisser-déposer. Boutons **+ Page** et **+ Section**.
- **Éléments disponibles** : variables et voies pas encore placées ; glissez-les sur une section (ou double-cliquez).
- **Aperçu** : en mode **Édition**, un clic sélectionne un widget ou une section et les widgets se déplacent à la souris ; en mode **Essai**, ils réagissent comme sur l'automate. Largeurs téléphone / tablette / écran.
- **Propriétés** (à droite) : nom et mot de passe de page ; titre, disposition et mot de passe de section ; libellé, type de widget et réglages (couleur, clignotement, bornes, pas, unité, décimales, choix de liste, seuils d'alarme...). Le bouton  revient à la valeur calculée.

Tant que rien n'est modifié, la disposition est **automatique** (recalculée à chaque nouvelle variable). La première modification la fige ; « Revenir à la disposition automatique » abandonne les réglages.



Étape 5 — éditeur visuel : structure, aperçu réel et propriétés du widget sélectionné.

Étape 6 — Simulation

Le projet tourne dans le navigateur avec le même moteur que le code C++ généré : grafcet animé (étapes actives et leur durée), **forçage des entrées**, commandes de l'écran (appui bref ou bloqué), paramètres, sorties, modèle simplifié du servo (déplacement, prise d'origine, jog, alarmes), messages et journal. Lecture continue ($\times 0,25$ à $\times 20$), cycle par cycle (**1 cycle**) ou par seconde (**1 s**) ; **Réinitialiser** repart de zéro.

Simulation

Le projet tourne ici exactement comme la classe C++ générée : même grafcet, mêmes modes de marche, même ordre de cycle (10 ms). Forcez les entrées, utilisez les commandes et observez les sorties.

Lancer **1 cycle** **1 s** **Réinitialiser** Vitesse $\times 1$

CYCLE Étapes actives : X1 4.02 s

Entrées (forçage)

- AU OK ☒
- AU_OK ☒
- Départ cycle ☒
- Pièce présente ☒
- Pièce ☒
- Vérin sorti ☐
- Verin_sorti ☐

Commandes de l'écran

- Acquitter ☐ bloquer
- Acquit ☐ bloquer
- Arrêt ☐ bloquer
- Jog ☐ bloquer
- Jog plus ☐ bloquer
- JogP ☐ bloquer
- Jog moins ☐ bloquer
- JogM ☐ bloquer

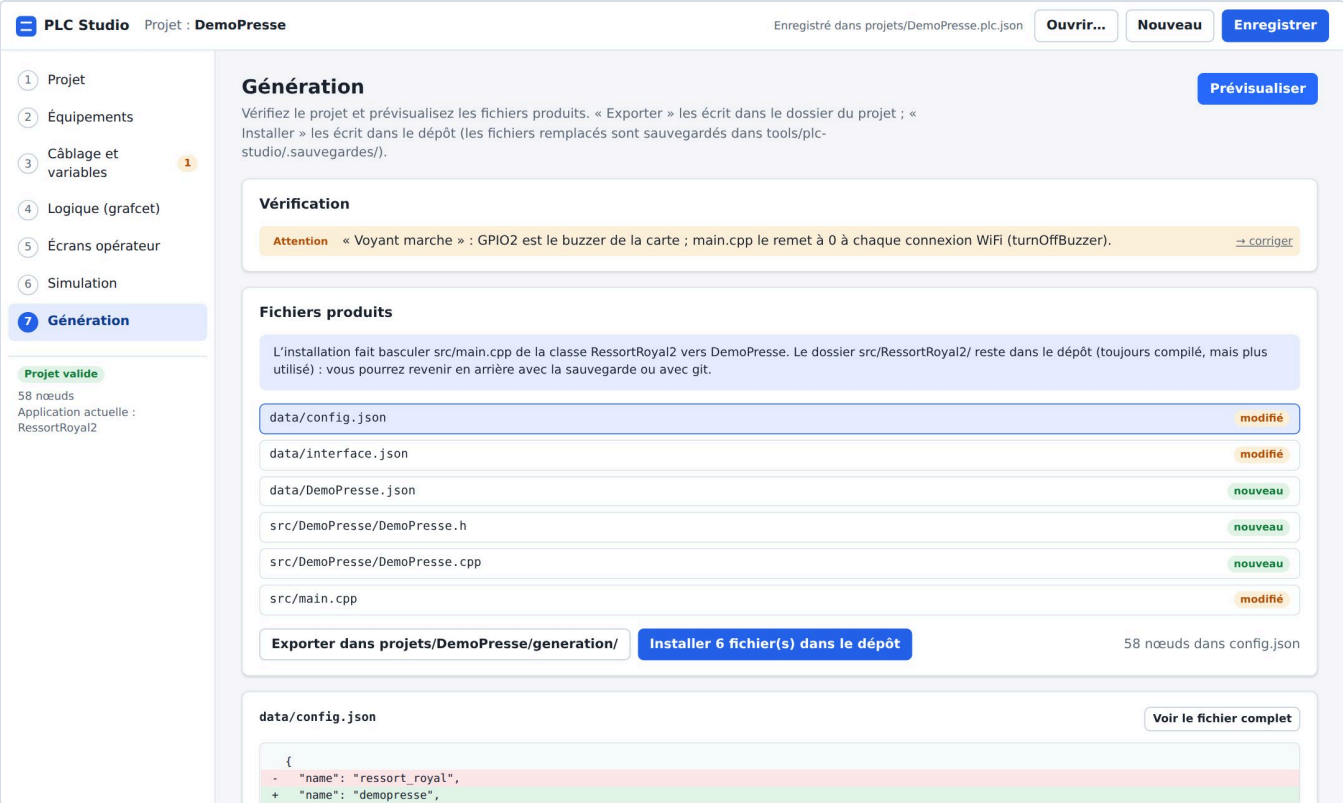
Sorties

Étape 6 — simulation : mode CYCLE, étape 1 active depuis 1 s, entrées forcées.

Étape 7 — Génération

Prévisualiser vérifie le projet et liste les fichiers produits (nouveau / modifié) avec les différences par rapport au dépôt. Deux façons d’écrire :

- **Exporter** : écrit dans `projets/<Nom>/generation/` , sans toucher au firmware ;
- **Installer** : écrit dans le dépôt et sauvegarde d’abord les fichiers remplacés dans `tools/plc-studio/.sauvegardes/<date>/` . Disponible seulement si le projet est sans erreur ; l’outil refuse d’écraser un dossier `src/<Nom>/` qu’il n’a pas généré.



Étape 7 — vérification, fichiers produits et différences.

Fichier produit	Contenu
data/config.json	Réseau, options, sections et nœuds (identifiants calculés).
data/interface.json	Pages, sections et widgets de l’écran opérateur.
data/<Nom>.json	Valeurs initiales des paramètres sauvegardés.
src/<Nom>/<Nom>.h / .cpp	Classe d’application dérivée de <code>BusinessLogic</code> (modes de marche + grafcet).
src/main.cpp	Bascule de la classe actuelle (ex. <code>RessortRoyal2</code>) vers <code><Nom></code> .
data/hardware/..._noRTC.json	Variante sans horloge, seulement si l’option horloge est décochée.

5. Mettre le projet sur l'automate

- 1 Dans PLC Studio, étape **Génération** : **Prévisualiser**, vérifiez les différences, puis **Installer**.
- 2 Compilez et téléversez le firmware avec PlatformIO (VS Code : tâches PlatformIO de l'environnement `quatre_mb_huge`, ou en ligne de commande depuis la racine du dépôt) :

```
pio run -e quatre_mb_huge          # compiler
pio run -e quatre_mb_huge -t upload # téléverser le firmware
pio run -e quatre_mb_huge -t uploadfs # téléverser data/ (config.json,
interface.json...)
```

- 3 Ouvrez l'interface de l'automate dans le navigateur et vérifiez les écrans et les entrées/sorties.
- 4 Enregistrez le projet (`projets/<Nom>.plc.json`) et les fichiers générés dans git.

Revenir en arrière Les fichiers remplacés sont dans `tools/plc-studio/.sauvegardes/<date>/` ; vous pouvez aussi annuler avec git. L'ancienne application (`src/RessortRoyal2/`) reste dans le dépôt et reste compilée : supprimez-la ou excluez-la si la place en flash manque.

6. Aide-mémoire de la logique

Réceptivités et expressions

Écriture	Signification
Depart ET NON Porte	Opérateurs logiques (aussi AND / OR / NOT ou && !).
FM(Bouton) ↑Bouton	Front montant (FD / ↓ : front descendant).
X3	Étape 3 active.
X3.t >= 5s ou t/X3/5s	Étape 3 active depuis 5 s (unités : ms, s, min, h ; un nombre seul comparé à X3.t est en ms).
Compteur >= Nb_pieces	Comparaisons : = <> < <= > >=
(Cible + 100) * 2	Arithmétique : + - * / % (division entière entre entiers).
MIN(a, b) MAX(a, b) ABS(a)	Fonctions.
Recette = "A"	Texte (= et <> uniquement).
VRAI FAUX	Constantes.

Actions d'étape

Action	Effet
Continue (N)	Vraie tant que l'étape est active (condition possible).
Mise à 1 mémorisée (S)	Reste à 1 jusqu'à une action R.
Mise à 0 mémorisée (R)	Annule une action S.
Affecter à l'activation	cible := valeur, une fois.
Affecter en continu	cible := valeur à chaque cycle.
Incrémenter / Décrémenter	+1 / -1 à l'activation.
Servo : aller à	Position (impulsions) et vitesse (0-15).
Servo : prise d'origine	À l'activation.
Servo : arrêt	Arrêt immédiat du mouvement.
Message à l'écran	Notification à l'activation.

Propriétés du servo

Symbole	Signification
Servo.pret	Servo prêt (alimenté, sans alarme).
Servo.enPosition	Dernier déplacement terminé.
Servo.enMouvement	Déplacement en cours.
Servo.origineFaite	Prise d'origine faite.
Servo.alarme	Alarme active.
Servo.position	Position actuelle (impulsions).

Cycle d'exécution (10 ms) capture des fronts → franchissement simultané des transitions validées, répété tant que possible (recherche de stabilité, 16 itérations au plus) avec exécution des actions à l'activation → actions continues et sorties. Une étape franchie dans le même cycle n'active pas ses actions continues ; un front ne vaut qu'une fois par cycle ; une sortie pilotée par N/S/R vaut le OU de toutes ses actions.

7. Règles appliquées automatiquement

PLC Studio vérifie et applique les contraintes du firmware, qui sont autant de causes de blocage au démarrage quand les fichiers sont écrits à la main :

- identifiant de nœud = CRC-32 de "`<id><nom de section>`", sans collision ;
- chaque widget de `interface.json` pointe vers un nœud existant de `config.json` ;
- nom de section + nom de nœud < 79 octets ; nom réseau sans espace ; 10 réseaux WiFi au maximum ;
- l'automate (porteur du bus RS485) est déclaré en premier ; adresses Modbus de 1 à 247 ;
- une seule instance SureServo ; toute consigne de position est bornée par la course maximale ; le jog est rafraîchi à chaque cycle ;
- une entrée « inversée » n'est pas inversée une seconde fois à l'écran ;
- les voies des modules Waveshare (un nœud par module) sont recopiées dans des nœuds « Miroirs ES » pour être affichées correctement ;
- les paramètres sont relus au démarrage depuis `data/<Nom>.json` , bornés par leurs min/max et sauvegardés quand ils changent.

8. Limites connues et dépannage

Limites connues

- **Mode headless** : la clé `execution_mode` est écrite mais pas encore lue par le firmware ; la logique ne tourne qu'avec un écran connecté. La correction de `main.cpp` est prévue.
- **Compilation ESP32** : le C++ généré est vérifié par l'outil (syntaxe, contre les vrais en-têtes du dépôt) ; la compilation complète se fait avec PlatformIO sur le poste.
- **Servo** : un seul SureServo par projet ; le simulateur utilise un modèle simplifié (vitesses et temps indicatifs).
- **Mode « Essai » des écrans** : les widgets réagissent, mais la logique du grafset n'y tourne pas (utilisez l'étape Simulation).
- **Import** : un projet écrit à la main (`config.json` + classe C++) ne peut pas être importé.
- Sur la carte Kincony A8S, GPIO2 (buzzer) est remis à 0 par `main.cpp` à chaque connexion WiFi : évitez d'y câbler une sortie.

Dépannage

Symptôme	Solution
« node » n'est pas reconnu	Installer Node.js (LTS) puis rouvrir le terminal.
Port 8090 déjà utilisé (EADDRINUSE)	Lancer avec <code>--port 8091</code> et ouvrir <code>http://localhost:8091</code> .
La page ne s'ouvre pas	Vérifier que le terminal est toujours ouvert et que l'adresse est bien <code>localhost</code> .
Bouton « Installer » grisé	Corriger les erreurs signalées dans la colonne de gauche (lien « → corriger »).
Installation refusée sur <code>src/<Nom>/</code>	Le dossier n'a pas été généré par l'outil : choisir un autre nom de projet ou déplacer le dossier.
L'automate ne démarre plus après installation	Vérifier que firmware et data/ ont tous deux été téléversés ; sinon restaurer la sauvegarde.

9. Pour les développeurs

```
cd tools/plc-studio
npm test          # tests unitaires (identifiants, langage, grafcet, simulateur, écrans...)
npm run check:cpp # vérification g++ -fsyntax-only du C++ généré (avec et sans servo)
```

Ajouter un équipement au catalogue :

1. créer (ou réutiliser) le fichier `data/hardware/<Nom>.json` lu par le firmware ;
2. ajouter une entrée dans `tools/plc-studio/shared/catalog.js` (fichier matériel, rôle, adresse Modbus par défaut, groupes de voies avec leur type de section et leurs identifiants) ;
3. lancer `npm test` : il vérifie que chaque identifiant généré existe dans le fichier matériel.

Organisation du code : `server/` (serveur HTTP local sans dépendance), `shared/` (modèle, génération, langage, grafcet, simulateur — partagés entre navigateur et tests), `public/` (interface), `public/hmi/` (faux automate pour l'aperçu des écrans), `test/`.